

Date :13.10.2025

Aim:

To implement a simple feedforward neural network with one hidden layer to solve the XOR problem.

Procedure:

1. Initialize weights and biases randomly for input-to-hidden and hidden-to-output layers.
2. Use sigmoid activation function and its derivative for non-linear transformations.
3. Perform forward pass: compute activations of hidden and output layers.
4. Compute error between network output and expected output.
5. Perform backpropagation to calculate gradients for output and hidden layers.
6. Update weights and biases using gradient descent.
7. Repeat for a fixed number of epochs to minimize error.
8. Test the trained network on all input combinations.

Program coding:

```
import math

import random

# Sigmoid activation and its derivative

def sigmoid(x):

    return 1 / (1 + math.exp(-x))

def sigmoid_derivative(x):

    return x * (1 - x)

# Training dataset: XOR problem

training_inputs = [

    [0, 0],
```

```

[0, 1],
[1, 0],
[1, 1]
]

training_outputs = [
    [0],
    [1],
    [1],
    [0]
]

# Initialize network parameters

input_neurons = 2

hidden_neurons = 2

output_neurons = 1

learning_rate = 0.5

# Initialize weights randomly

random.seed(42)

weights_input_hidden = [[random.uniform(-1, 1) for _ in range(hidden_neurons)] for _ in
range(input_neurons)]

weights_hidden_output = [random.uniform(-1, 1) for _ in range(hidden_neurons)]

# Initialize biases

bias_hidden = [random.uniform(-1, 1) for _ in range(hidden_neurons)]

bias_output = random.uniform(-1, 1)

# Training loop

```

```

for epoch in range(10000):

    total_error = 0

    for inputs, expected in zip(training_inputs, training_outputs):

        # ---Forward pass ---

        # Hidden layer activations

        hidden_layer_input = []

        for j in range(hidden_neurons):

            activation = bias_hidden[j]

            for i in range(input_neurons):

                activation += inputs[i] * weights_input_hidden[i][j]

            hidden_layer_input.append(sigmoid(activation))

        # Output layer activation

        output_activation = bias_output

        for j in range(hidden_neurons):

            output_activation += hidden_layer_input[j] * weights_hidden_output[j]

        output = sigmoid(output_activation)

        # --- Calculate error ---

        error = expected[0] - output

        total_error += error**2

        # --- Backward pass ---

        # Output layer delta

        delta_output = error * sigmoid_derivative(output)

        # Hidden layer deltas

        delta_hidden = []

        for j in range(hidden_neurons):

```

```

        delta = delta_output * weights_hidden_output[j] *
sigmoid_derivative(hidden_layer_input[j])

        delta_hidden.append(delta)

# --- Update weights and biases ---

# Update weights hidden → output

for j in range(hidden_neurons):

    weights_hidden_output[j] += learning_rate * delta_output * hidden_layer_input[j]

    bias_output += learning_rate * delta_output

# Update weights input → hidden

for i in range(input_neurons):

    for j in range(hidden_neurons):

        weights_input_hidden[i][j] += learning_rate * delta_hidden[j] * inputs[i]

    for j in range(hidden_neurons):

        bias_hidden[j] += learning_rate * delta_hidden[j]

# Print error every 1000 epochs

if epoch % 1000 == 0:

    print(f'Epoch {epoch} - Error: {total_error:.4f}')

# Test trained network

print("\nTesting trained network:")

for inputs in training_inputs:

    hidden_layer_input = []

    for j in range(hidden_neurons):

        activation = bias_hidden[j]

        for i in range(input_neurons):

            activation += inputs[i] * weights_input_hidden[i][j]

        hidden_layer_input.append(sigmoid(activation))

```

```
output_activation = bias_output

for j in range(hidden_neurons):

    output_activation += hidden_layer_input[j] * weights_hidden_output[j]

output = sigmoid(output_activation)

print(f'Input: {inputs} => Output: {output:.4f}')
```

Output:

Epoch 0 - Error: 1.3576

Epoch 1000 - Error: 0.2632

Epoch 2000 - Error: 0.1235

Epoch 3000 - Error: 0.0739

Epoch 4000 - Error: 0.0492

Epoch 5000 - Error: 0.0364

Epoch 6000 - Error: 0.0281

Epoch 7000 - Error: 0.0224

Epoch 8000 - Error: 0.0183

Epoch 9000 - Error: 0.0150

Testing trained network:

Input: [0, 0] => Output: 0.0334

Input: [0, 1] => Output: 0.9660

Input: [1, 0] => Output: 0.9662

Input: [1, 1] => Output: 0.0376

Result:

The multi-layer neural network successfully learned the XOR function, achieving low error and producing outputs close to expected binary results.